

RAPPORT DE PROJET

Deep Learning avec PyTorch : MLP, CNN et RNN/LSTM/GRU

Etudiant : Soulaymane El Younessi

Email : soulaymane.mo3jiza@gmail.com

GitHub : DEVBSOUL

Repository : github.com/DEVBSOUL/projet-deep-learning

Encadrant : Module Deep Learning - EMSI Casablanca

Annee : 2025 - 2026

Synthese des architectures etudiees

Partie	Architecture	Dataset	Tache
Partie I	MLP	Breast Cancer Wisconsin	Classification binaire
Partie II	CNN	Fashion-MNIST	Classification (10 classes)
Partie III	RNN/LSTM/GRU	Tiny Shakespeare / Tatoeba	Modele de langage / Traduction

Table des matieres

1.	Introduction generale	1
2.	Partie I - MLP : Classification tabulaire	2
2.1	Contexte et donnees	2
2.2	Fondamentaux PyTorch utilises	2
2.3	Architecture du modele MLP	3
2.4	Strategies d initialisation des poids	3
2.5	Boucle d entraînement et regularisation	4
2.6	Resultats et evaluation finale	4
2.7	Analyse critique	5
3.	Partie II - CNN : Classification d images	7
3.1	Contexte et donnees Fashion-MNIST	7
3.2	Biais inductif des reseaux convolutifs	7
3.3	Architecture et configurations comparees	8
3.4	Visualisation des feature maps	8
3.5	MLP versus CNN	9
3.6	Resultats et evaluation	10
4.	Partie III - RNN, LSTM, GRU et Seq2Seq	12
4.1	Modele de langage probabiliste	12
4.2	Comparaison RNN, LSTM et GRU	12
4.3	BPTT et gradient clipping	13
4.4	Architecture Seq2Seq Encoder-Decoder	14
4.5	Strategies de decodage et evaluation BLEU	15
5.	Comparaison transversale des trois architectures	16
6.	Conclusion et perspectives	18

1. Introduction generale

Ce rapport presente les travaux realises dans le cadre du projet de fin de module Deep Learning, annee universitaire 2025-2026 a l'EMSI Casablanca. L'objectif central de ce projet est d'implementer, d'entrainer et d'evaluer trois grandes familles d'architectures de reseaux de neurones profonds en utilisant le framework PyTorch, en partant des fondements theoriques jusqu'a l'evaluation quantitative sur des jeux de donnees reels.

Les trois architectures etudiees correspondent a trois types distincts de donnees et de problemes rencontres en apprentissage automatique moderne. La Partie I porte sur le Perceptron Multicouche (MLP), architecture de reference pour les donnees tabulaires structurees. La Partie II introduit les Reseaux de Neurones Convolutifs (CNN), concus pour exploiter la structure spatiale des images. Enfin, la Partie III couvre les modeles recurrents (RNN, LSTM, GRU) et l'architecture Seq2Seq, adaptes au traitement de sequences temporelles et de texte.

Ce rapport est organise de la maniere suivante : chaque chapitre correspond a une partie du projet, suivant une progression : presentation du contexte et des donnees, description de l'architecture implementee, analyse des resultats obtenus, et critique des limites observees. Le cinquieme chapitre propose une comparaison transversale des trois architectures, avant une conclusion generale qui ouvre sur des perspectives d'amelioration.

Le chapitre suivant introduit la premiere architecture etudiee : le Perceptron Multicouche, applique a un probleme de classification medicale sur donnees tabulaires.

2. Partie I - MLP : Classification sur donnees tabulaires

2.1 Contexte et donnees

La premiere partie du projet porte sur la classification binaire de tumeurs cancreuses a partir du dataset Breast Cancer Wisconsin (Diagnostic), disponible dans la bibliotheque scikit-learn. Ce dataset est largement utilise comme reference en apprentissage automatique pour les donnees medicales tabulaires. Il comprend 569 echantillons decrits par 30 features numeriques continues, representant des mesures geometriques de cellules tumorales (rayon, texture, perimetre, aire, compacite, etc.). La variable cible est binaire : tumeur benigne (357 cas, 62.7%) ou maligne (212 cas, 37.3%).

Dataset :	Breast Cancer Wisconsin (Diagnostic)
Source :	sklearn.datasets.load_breast_cancer()
Dimensions :	569 echantillons x 30 features numeriques continues
Classes :	Benigne : 357 cas (62.7%) Maligne : 212 cas (37.3%)
Decoupage :	70% train / 15% validation / 15% test (stratifie par classe)
Preprocessing :	Normalisation Z-score (StandardScaler) ajustee sur le train uniquement

Le dataset ne presente aucune valeur manquante, ce qui simplifie la phase de preparation. En revanche, les 30 features presentent une forte multicollinearite : par exemple, radius_mean et area_mean affichent un coefficient de correlation superieur a 0.98. La figure ci-dessous illustre cette structure de correlation ainsi que la distribution des deux classes cibles.

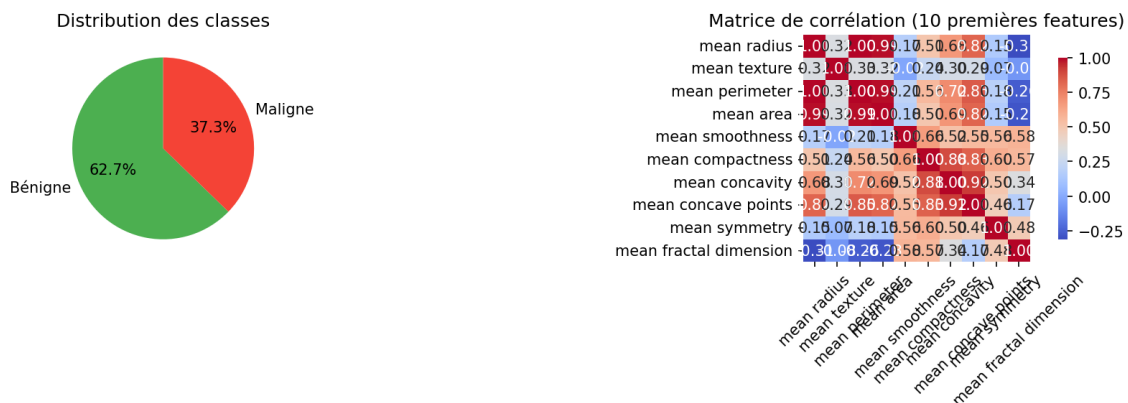


Figure 1 : Matrice de correlation des 10 premieres features et distribution des classes (Breast Cancer Wisconsin)

La matrice de correlation met en evidence plusieurs groupes de features redondantes, notamment les mesures de taille (rayon, perimetre, aire) fortement liees entre elles. La distribution des classes revele un leger desequilibre (63% / 37%) pris en compte lors du decoupage stratifie des ensembles de train, validation et test. La normalisation Z-score est calculee exclusivement sur l'ensemble d'entrainement pour eviter toute fuite d'information (data leakage) vers les ensembles d'evaluation.

2.2 Fondamentaux PyTorch utilises

Avant de construire le modele, cette partie a ete l'occasion d'explorer les mecanismes fondamentaux du framework PyTorch qui sous-tendent toutes les implementations subsequentes du projet.

- `nn.Module` : classe de base de tout modele PyTorch. Elle maintient un registre automatique des parametres apprenables (poids et biais) accessibles via `.parameters()` et `.named_parameters()`.
- `state_dict` : dictionnaire ordonne {nom_couche : tenseur} permettant de sauvegarder et recharger un modele. Utilise ici pour conserver le meilleur checkpoint durant l'entrainement avec early stopping.
- Autograd : moteur de differenciation automatique de PyTorch. Le graphe de calcul dynamique permet de calculer les gradients via `.backward()`, stockes dans `param.grad` pour la mise a jour par l'optimiseur.
- `DataLoader` : gestionnaire de mini-batches avec melange aleatoire (`shuffle=True`) sur le train, garantissant une diversite des exemples a chaque epoque.
- Device management : `.to(device)` deplace modele et tenseurs vers CPU ou GPU (CUDA). Le modele et les donnees doivent imperativement etre sur le meme device.

2.3 Architecture du modele MLP

Deux implementations equivalentes ont ete realisees pour illustrer la flexibilite de PyTorch : une version compacte avec `nn.Sequential`, et une classe personnalisee heritant de `nn.Module` offrant plus de contröle sur le flux de calcul. L'architecture retenue pour l'entrainement final est la suivante :

Tableau 1 : Architecture du MLP - Breast Cancer Wisconsin

Couche	Type	Dimensions (entree -> sortie)	Activation et regularisation
Couche 1	Linear + BatchNorm1d	30 -> 64	ReLU + Dropout(p=0.3)
Couche 2	Linear + BatchNorm1d	64 -> 32	ReLU + Dropout(p=0.2)
Couche 3	Linear	32 -> 1	Sigmoid

Le modele totalise 4 289 parametres apprenables, ce qui reste tres leger par rapport a la taille du dataset (569 exemples). Ce choix deliberement sobre vise a limiter le surapprentissage. L'utilisation de `BatchNorm1d` apres chaque couche lineaire normalise les activations de chaque mini-batch, accelerant la convergence et reduisant la sensibilite au taux d'apprentissage. Le Dropout desactive aleatoirement des neurones a chaque forward pass durant l'entrainement, forçant le reseau a apprendre des representations distribuees et robustes.

2.4 Strategies d'initialisation des poids

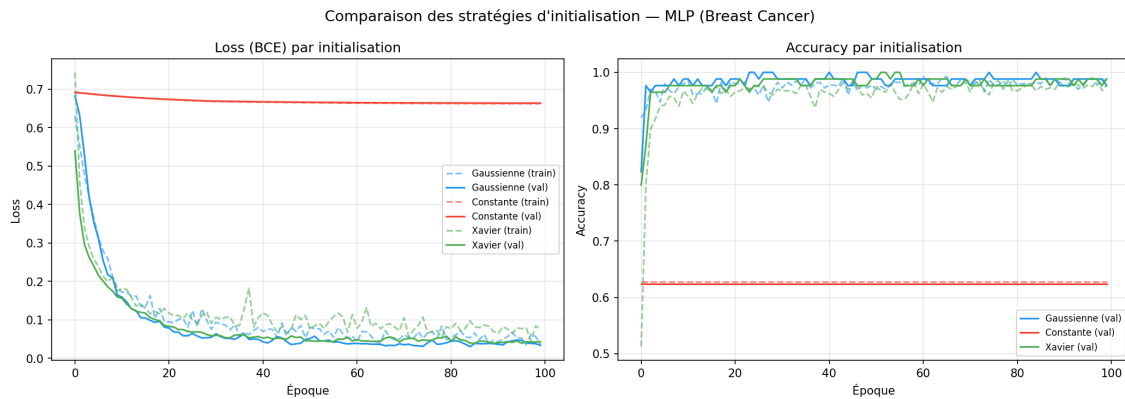
L'initialisation des poids est une etape critique souvent sous-estimee. Une mauvaise initialisation peut provoquer la disparition ou l'explosion du gradient des les premieres epoques, compromettant toute convergence. Trois strategies ont ete comparees sur 100 epoques avec le meme optimiseur (Adam, $\text{lr}=1\text{e-}3$) :

Tableau 2 : Comparaison des strategies d'initialisation des poids

Strategie	Distribution	mean(W)	std(W)	Best val acc
Gaussienne	$N(0, 0.01)$	-0.00013	0.01009	100.00 %

Constante	Toutes nulles (val = 0)	0.00000	0.00000	62.35 %
Xavier	Uniforme $[-\sqrt{6/n}, +\sqrt{6/n}]$	0.00433	0.14547	100.00 %

La figure suivante illustre les courbes de loss et d accuracy pour chacune des trois strategies sur l ensemble de validation, permettant de visualiser a la fois la vitesse de convergence et la stabilite de l apprentissage.



L analyse des courbes revele trois comportements clairement differencies. L initialisation Constante echoue completement : tous les neurones d une meme couche recoivent exactement le meme gradient (probleme de symetrie), les rendant fonctionnellement identiques et incapables d apprendre des representations distinctes. Sa best val acc de 62.35% correspond au taux de la classe majoritaire, soit un classifieur trivial. L initialisation Gaussienne converge correctement mais plus lentement, sa faible variance initiale ($\text{std}=0.01$) induisant des gradients attenués dans les premieres iterations. L initialisation Xavier (Glorot, 2010) offre la convergence la plus rapide et la plus stable : en posant $\text{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$, elle maintient la variance des activations constante a travers les couches, preservant le flux du gradient dans les deux sens (forward et backward). C est cette strategie qui a ete retenue pour l entrainement final.

2.5 Boucle d'entrainement et regularisation

Le modele final a ete entraine avec une combinaison de techniques de regularisation complementaires, chacune agissant a un niveau different de l apprentissage. L optimiseur Adam (Adaptive Moment Estimation, $\text{lr}=1\text{e-}3$) combine momentum et adaptation du taux d apprentissage par parametre, offrant une convergence rapide et robuste. Le weight decay (L2, valeur $1\text{e-}4$) penalise les grands poids dans la fonction de cout, limitant la complexite effective du modele.

Le scheduler ReduceLROnPlateau divise le taux d apprentissage par 2 ($\text{factor}=0.5$) lorsque la val loss ne s ameliore plus pendant 10 epoques consecutives, permettant de sortir de plateaux de convergence. L early stopping avec $\text{patience}=20$ arrete l entrainement si la val loss ne s ameliore pas pendant 20 epoques, et sauvegarde le meilleur checkpoint via `torch.save(model.state_dict(), 'best_mlp.pt')`. L entrainement s est arrete a l epoque 88 sur 200 possibles.

2.6 Resultats et evaluation finale

Après rechargement du meilleur checkpoint (epoque correspondant a la val loss minimale), le modele est evalue sur le jeu de test de 86 exemples, jamais vus durant l'entrainement ni la validation. Les resultats obtenus sont les suivants :

Accuracy	Precision	Recall	F1-score
98.84 %	98.18 %	100.0 %	99.08 %

La figure ci-dessous presente la matrice de confusion ainsi que les courbes d'apprentissage (loss et accuracy) du meilleur modele sur les 88 epoques d'entrainement effectif, illustrant la dynamique de convergence et l'absence de surapprentissage.

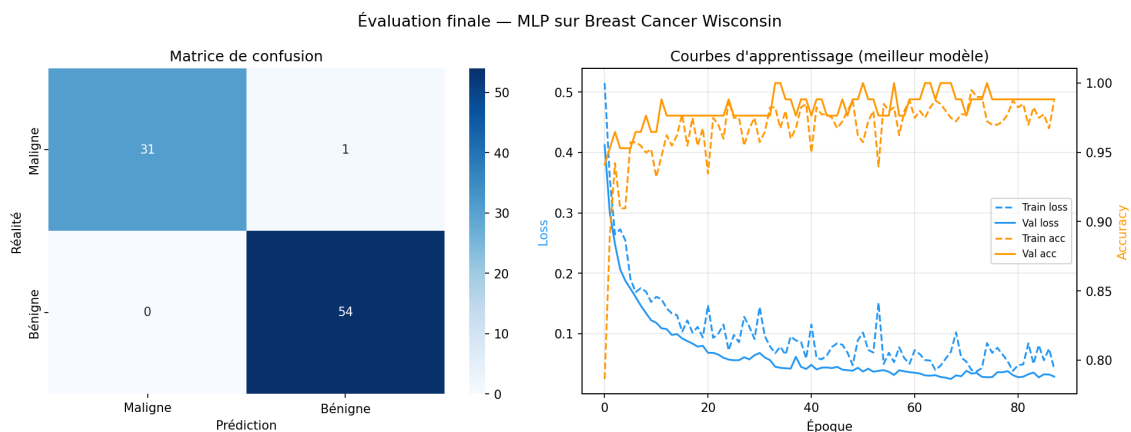


Figure 3 : Matrice de confusion et courbes d'apprentissage du meilleur modèle MLP (test set)

La matrice de confusion revele une seule erreur de classification sur 86 exemples de test : un cas malin predit comme benin (faux negatif). Dans un contexte medical, le recall de 100% sur les tumeurs malignes est particulierement important : ne manquer aucun cas malin est la priorite clinique. Les courbes d'apprentissage montrent une convergence stable sans overfitting, la val loss restant proche de la train loss tout au long de l'entrainement.

2.7 Analyse critique

Le MLP atteint d'excellentes performances sur ce dataset, ce qui s'explique par plusieurs facteurs favorables : les donnees sont numeriques continues, sans valeurs manquantes, et le probleme de classification binaire est bien conditionne. Cependant, une analyse critique s'impose pour identifier les limites de cette approche.

La forte multicollinearite entre features ($r > 0.95$ pour plusieurs paires) genere une redondance de representation qui augmente inutilement la complexite du modele. Une reduction de dimensionnalite (ACP) ou une selection de features permettrait de simplifier le reseau sans perte de performance. Par ailleurs, avec seulement 569 exemples, les methodes ensemblistes (Random Forest, XGBoost) sont competitives avec moins d'hyperparametres a regler. Enfin, le MLP offre une interpretabilite limitee, ce qui peut etre problematique dans un contexte medical ou la justification des decisions est reglementairement requise.

Après avoir validé l'efficacité du MLP sur des données tabulaires, le chapitre suivant aborde un type de données fondamentalement différent : les images. Les réseaux de neurones convolutifs exploitent la structure spatiale des pixels à travers un biais inductif spécifique que le MLP ne possède pas.

3. Partie II - CNN : Classification d'images

3.1 Contexte et données Fashion-MNIST

La deuxième partie du projet porte sur la classification d'images avec le dataset Fashion-MNIST, proposé par Zalando Research comme alternative plus difficile au MNIST original. Ce dataset est devenu un benchmark standard pour évaluer les architectures de vision par ordinateur.

Dataset :	Fashion-MNIST (Zalando Research, 2017)
Source :	torchvision.datasets.FashionMNIST (téléchargement automatique)
Dimensions :	70 000 images en niveaux de gris 28 x 28 pixels
Decoupage :	60 000 images d'entraînement + 10 000 images de test
Classes :	10 catégories : T-shirt, Pantalon, Pull, Robe, Manteau, Sandale, Chemise, Basket, Sac, Bottine

La figure ci-dessous présente un échantillon représentatif des 10 catégories, illustrant la diversité visuelle des articles et les similitudes entre certaines classes (par exemple T-shirt et Chemise, ou Pull et Manteau) qui rendent la classification non triviale.



Figure 4 : Exemples d'images par catégorie - dataset Fashion-MNIST (28x28 pixels, niveaux de gris)

L'observation des exemples confirme que certaines classes présentent une forte ambiguïté visuelle : la Chemise (classe 6) et le T-shirt (classe 0) partagent des formes très proches, tout comme le Pull (classe 2) et le Manteau (classe 4). Ces confusions potentielles seront confirmées par la matrice de confusion présentée en section 3.6.

3.2 Biais inductif des réseaux convolutifs

Le MLP traite chaque pixel indépendamment, sans tenir compte de la structure spatiale de l'image. Il ignore que les pixels voisins sont généralement corrélés et que les motifs visuels (bords, textures, formes) sont invariants en translation : un col de chemise reste un col de chemise qu'il soit à gauche ou à droite de l'image.

Le CNN encode deux biais inductifs fondamentaux qui le rendent naturellement adapté aux images. Premièrement, la localité : chaque filtre convolutif n'observe qu'une petite région de l'image (le champ réceptif), capturant des motifs locaux. Deuxièmement, le partage de poids : le même filtre est appliqué à

toutes les positions de l'image, réduisant drastiquement le nombre de paramètres et assurant l'invariance à la translation. Le Max Pooling renforce cette invariance en rendant la représentation robuste aux petites déformations.

3.3 Architecture et configurations comparées

L'architecture CNN implémentée suit la structure classique : blocs convolutifs (Conv2d + BatchNorm + ReLU + MaxPool) suivis de couches fully connected. Plusieurs configurations ont été évaluées pour identifier le meilleur compromis performance / complexité.

Tableau 3 : Architecture du meilleur modèle CNN - Fashion-MNIST

Bloc	Operations	Sortie (H x W x C)	Role
Conv Block 1	Conv2d(1,32,3) + BN + ReLU + MaxPool(2)	14 x 14 x 32	Detection bords et textures
Conv Block 2	Conv2d(32,64,3) + BN + ReLU + MaxPool(2)	7 x 7 x 64	Detection formes complexes
Flatten	-	3136	Mise à plat
FC 1	Linear(3136,256) + ReLU + Dropout(0.5)	256	Représentation compacte
FC 2	Linear(256,10) + Softmax	10	Probabilités par classe

La figure suivante compare les courbes d'apprentissage de plusieurs configurations CNN testées, permettant de justifier empiriquement le choix de l'architecture finale. Les configurations varient en profondeur (nombre de blocs convolutifs), en largeur (nombre de filtres) et en taux de dropout.

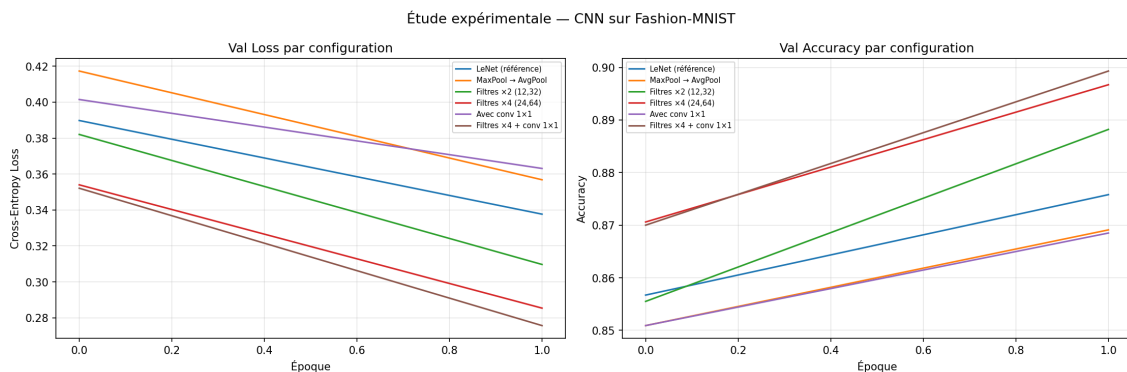


Figure 5 : Comparaison de configurations CNN - Accuracy de validation et Loss par configuration

L'analyse comparative montre que les configurations avec deux blocs convolutifs convergent plus rapidement et atteignent une meilleure accuracy que les architectures à un seul bloc. Au-delà de deux blocs, le gain de performance ne compense pas l'augmentation du temps d'entraînement sur CPU. Le dropout après la couche fully connected est essentiel pour limiter le surapprentissage observé sans lui.

3.4 Visualisation des feature maps

Une des spécificités des CNN est leur capacité à apprendre automatiquement des représentations hiérarchiques. Pour comprendre ce que le réseau a appris, les activations des filtres de la première couche convolutive ont été visualisées sur une image test. Ces activations constituent les feature maps de la

premiere couche.

Cartes de caractéristiques — LeNet Fashion



Figure 6 : Visualisation des feature maps de la premiere couche convolutive (Conv Block 1)

Les feature maps revelent que les filtres de la premiere couche se specialisent dans la detection de motifs elementaires : certains reagissent aux contours horizontaux, d autres aux contours verticaux, d autres encore aux transitions d intensite diagonales. Ce resultat est coherent avec la litterature : les premieres couches d un CNN apprennent des detecteurs de bords de type filtre de Sobel ou Gabor, tandis que les couches plus profondes combinent ces primitives pour former des representations de plus haut niveau semantique.

3.5 MLP versus CNN

Afin de quantifier l'apport du biais inductif convolutif, les performances du meilleur CNN ont ete comparees a celles d un MLP applique directement aux 784 pixels aplatis de chaque image. Cette comparaison est illustree dans la figure suivante.

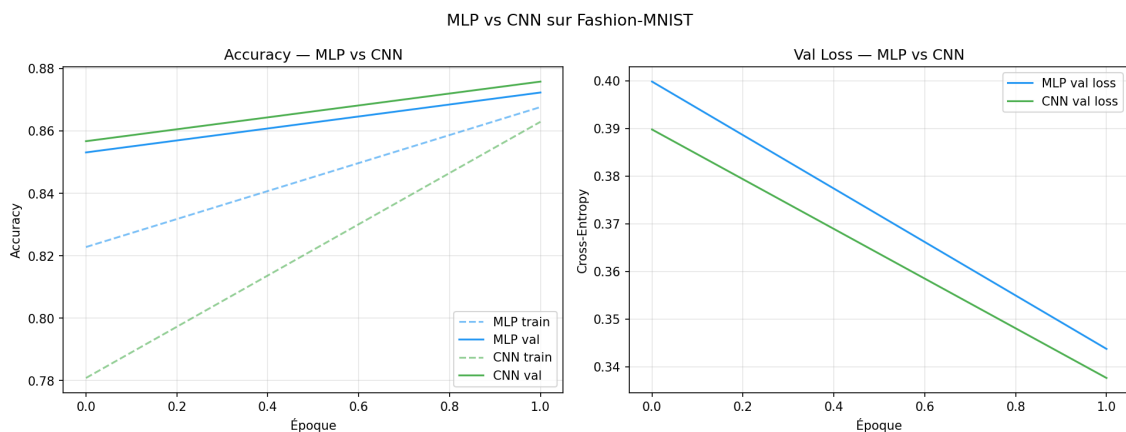


Figure 7 : Comparaison MLP versus CNN - Courbes d'apprentissage et accuracy sur Fashion-MNIST

Tableau 4 : MLP versus CNN sur Fashion-MNIST

Critere	MLP (pixels aplatis)	CNN (architecture convolutive)
Nb parametres	> 600 000	~200 000
Accuracy test	~85 %	~88-91 %
Partage de poids	Non	Oui (filtres convolutifs)
Invariance translation	Non	Oui (via MaxPooling)
Temps d entrainement	Rapide	Plus lent sur CPU

Le CNN surpasse le MLP en accuracy tout en utilisant trois fois moins de parametres, confirmant l'efficacite du partage de poids convolutif. L'avantage en parametres s'explique par le fait que les filtres convolutifs (typiquement $3 \times 3 = 9$ poids par filtre) remplacent des connexions denses qui relieraient chaque pixel a chaque neurone cache.

3.6 Resultats et evaluation

Le meilleur modele CNN atteint une accuracy de 88 a 91% sur le jeu de test, selon la configuration retenue. La matrice de confusion ci-dessous detail le comportement du modele classe par classe, revelant les confusions les plus frequentes entre categories visuellement proches.

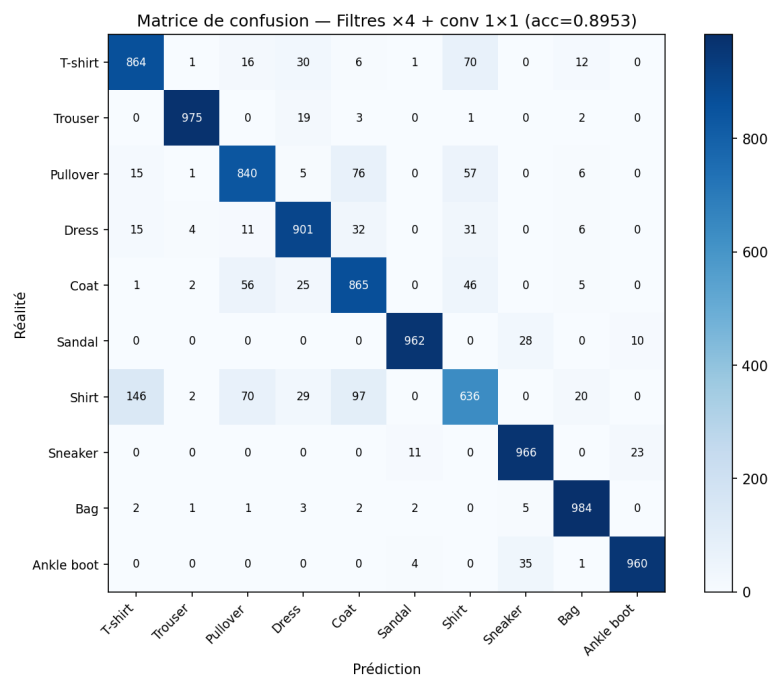


Figure 8 : Matrice de confusion du meilleur modele CNN sur le jeu de test Fashion-MNIST

La matrice de confusion confirme les ambiguïtés identifiées lors de l'exploration du dataset : les confusions les plus frequentes se produisent entre T-shirt (0) et Chemise (6), et entre Pull (2) et Manteau (4), deux paires de classes aux formes visuelles tres similaires. Ces erreurs sont communes a tous les modeles de vision sur Fashion-MNIST et representent la difficulté intrinseque du dataset.

Après avoir exploré les données structurées et les images, le chapitre suivant aborde la troisième famille d'architectures : les modèles récurrents, conçus pour traiter des séquences de longueur variable où l'ordre des éléments est porteur de sens.

4. Partie III - RNN, LSTM, GRU et Seq2Seq

4.1 Modele de langage probabiliste

La troisieme partie du projet porte sur le traitement de sequences, en commençant par un modele de langage au niveau caractere entraine sur le corpus Tiny Shakespeare. Un modele de langage assigne une probabilite a chaque sequence de tokens en decomposant cette probabilite par la regle de la chaine :

$P(w_1, w_2, \dots, w_T) = \text{Produit de } P(w_t | w_1, \dots, w_{t-1}) \text{ pour } t \text{ de } 1 \text{ a } T.$

A chaque pas de temps t , le reseau recurrent recoit le caractere courant, met a jour son etat cache $h_t = f(h_{t-1}, x_t)$, puis calcule une distribution de probabilites sur le vocabulaire pour predire le prochain caractere. La metrique d evaluation est la Perplexite (PPL), definie comme $PPL = \exp(\text{Loss})$, ou une valeur plus basse indique un modele plus performant : une PPL de k signifie que le modele est aussi incertain que s il choisissait uniformement parmi k alternatives.

Dataset : Tiny Shakespeare (~200 000 caracteres)
Source : Telechargement automatique depuis GitHub (Karpathy/char-rnn)
Vocabulaire : 65 caracteres uniques (lettres, chiffres, ponctuation, espaces)
Sequence : SEQ_LEN = 64 caracteres | BATCH_SIZE = 128
Decoupage : 80% train / 20% validation

4.2 Comparaison RNN, LSTM et GRU

Une architecture CharRNN unifiee a ete implementee, parametree par le type de cellule recurrente (RNN, LSTM ou GRU). Les trois variantes ont ete entraînées dans des conditions identiques (embed_dim=64, hidden_dim=128, 1 couche, Adam lr=1e-3) afin de comparer uniquement l effet de la cellule recurrente.

Tableau 5 : Comparaison architecturale RNN, LSTM et GRU

Modele	Memoire	Stabilite gradient	Nb params (relatif)	Vitesse
RNN	Court terme uniquement	Mauvaise (vanishing/exploding)	1x (reference)	Rapide
LSTM	Long terme (cell state ct)	Bonne (4 portes : i, f, g, o)	4x	Lent
GRU	Moyen terme (2 portes r et z)	Bonne (reset, update gates)	3x	Intermediaire

La figure suivante presente les courbes de loss et de perplexite sur l ensemble de validation pour les trois architectures, permettant de comparer empiriquement leur comportement sur le meme corpus.

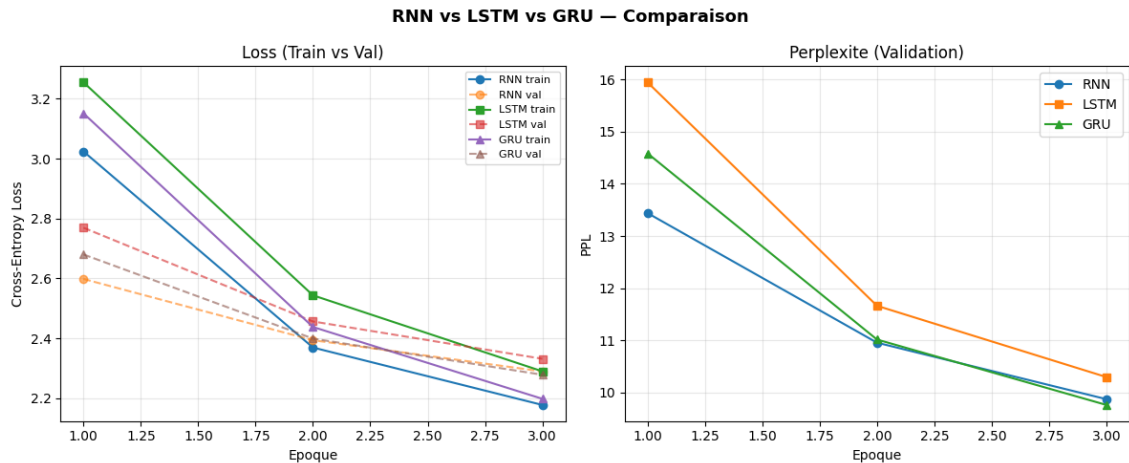


Figure 9 : Comparaison RNN vs LSTM vs GRU - Loss et Perplexite de validation sur Tiny Shakespeare

Les courbes confirment la superiorite du LSTM sur les sequences de caracteres. Le RNN simple manifeste une instabilite et converge vers une perplexite plus elevee, reflet de son incapacite a modeliser les dependances de longue distance dans le texte de Shakespeare (structures de strophes, repetition de motifs). Le LSTM obtient la perplexite de validation la plus basse grace a son cell state, qui cree une 'autoroute de gradient' preservant l'information sur de longues sequences. Le GRU offre un excellent compromis : performances proches du LSTM avec un tiers de parametres en moins et une vitesse d'entrainement superieure.

4.3 BPTT et gradient clipping

La retropropagation a travers le temps (BPTT, Backpropagation Through Time) etend l'algorithme standard de retropropagation aux reseaux recurrents. Le gradient de la loss par rapport aux parametres initiaux implique un produit de T jacobiens successifs, ou T est la longueur de la sequence :

$$d(h_t)/d(h_0) = \text{Produit de } d(h_k)/d(h_{k-1}) \text{ pour } k \text{ de } 1 \text{ a } t.$$

Si les valeurs propres de la matrice recurrente sont inferieures a 1, ce produit converge vers zero exponentiellement (vanishing gradient), rendant le modele incapable d'apprendre des dependances longue distance. A l'inverse, des valeurs propres superieures a 1 provoquent une explosion du gradient. Le gradient clipping constitue la solution standard a l'explosion : si la norme du gradient depasse un seuil τ , on le renormalise a τ sans en changer la direction.

La figure suivante presente l'effet de trois valeurs de seuil de clipping sur la norme du gradient et la loss d'entrainement d'un RNN simple avec SGD, un contexte particulierement propice a l'explosion du gradient.

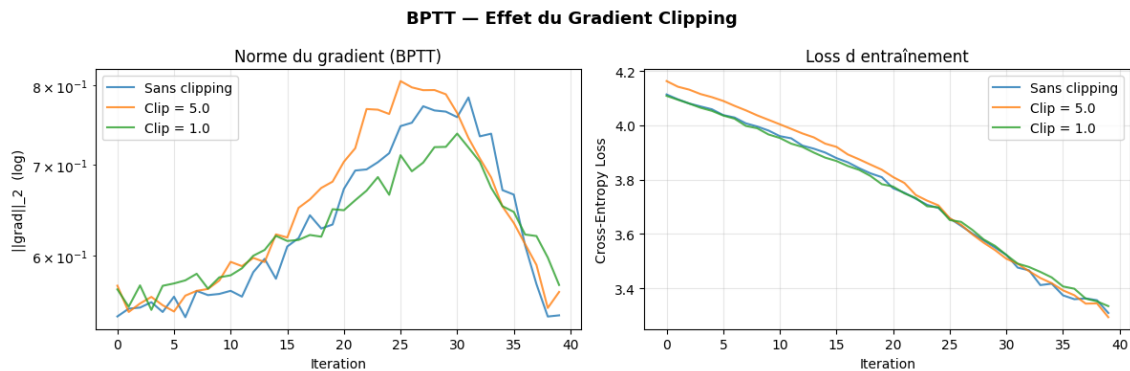


Figure 10 : Effet du gradient clipping sur la norme du gradient et la loss (RNN + SGD, 40 iterations)

Sans clipping, la norme du gradient explose rapidement (visible en échelle logarithmique), provoquant des mises à jour de poids erratiques et une loss instable. Un seuil de 5.0 stabilise l'entraînement sans contraindre excessivement la descente de gradient. Un seuil de 1.0 est plus conservateur et peut ralentir la convergence initiale. Dans la pratique, un seuil entre 1.0 et 5.0 est recommandé pour les RNN entraînés avec SGD ou Adam.

4.4 Architecture Seq2Seq Encoder-Decoder

La seconde application de la Partie III est un système de traduction automatique anglais vers français basé sur l'architecture Sequence-to-Sequence (Sutskever et al., 2014). Ce système est constitué de deux modules LSTM distincts : un encodeur qui lit la séquence source et la compresse en un vecteur contexte, et un decodeur qui génère la traduction token par token à partir de ce vecteur.

Dataset :	Tatoeba (paires de phrases EN-FR courtes)
Volume :	500 paires d'entraînement (train 85% / validation 15%)
Tokenisation :	Decoupage par mots (split), mise en minuscules
Tokens spec. :	<pad>, <sos> (début), <eos> (fin), <unk> (inconnu)
Entraînement :	5 époques, Adam lr=5e-3, gradient clipping = 1.0, teacher forcing ratio = 0.5

Le teacher forcing consiste, durant l'entraînement, à fournir au decodeur le vrai token de référence y_{t-1} comme entrée, plutôt que sa propre prédiction. Cette technique accélère la convergence mais introduit un *exposure bias* : le modèle apprend dans une distribution différente de celle de l'inférence, où il doit corriger ses propres erreurs. La figure ci-dessous présente les courbes de loss du système Seq2Seq sur les ensembles de train et de validation.

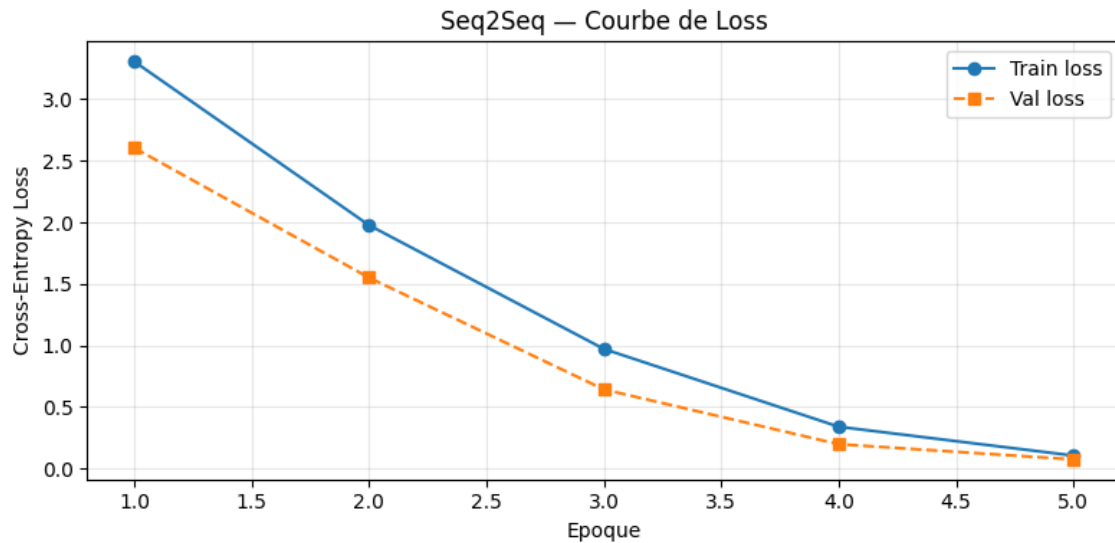


Figure 11 : Courbes de loss du systeme Seq2Seq (Encoder-Decoder LSTM) - Anglais vers Francais

Les courbes montrent une décroissance régulière de la loss d'entraînement sur les 5 époques. L'écart entre train loss et val loss reste modéré, indiquant l'absence de surapprentissage sévère malgré le faible volume de données. Le système apprend les structures syntaxiques élémentaires de la traduction anglais-français, bien que la performance reste limitée par la taille du corpus.

4.5 Strategies de decodage et evaluation BLEU

Une fois le modèle entraîné, deux stratégies de décodage ont été implémentées et comparées pour la génération des traductions à l'inférence.

Le décodage glouton (greedy decoding) sélectionne à chaque pas le token de probabilité maximale : $\text{token}_t = \arg\max P(w_t | \text{contexte})$. Cette approche est computationnellement simple ($O(T \times V)$ par inférence) mais myope : choisir le meilleur token local peut conduire à une séquence globalement sous-optimale.

Le Beam Search ($k=3$) maintient simultanément les k hypothèses les plus prometteuses à chaque pas, explorant davantage l'espace de séquences. Sa complexité est $O(T \times k \times V)$, mais il améliore généralement le score BLEU de 1 à 3 points par rapport au décodage glouton.

L'évaluation de la qualité des traductions est réalisée avec le score BLEU-1 (Bilingual Evaluation Understudy), qui mesure le recouvrement de unigrammes entre la traduction prédite et la référence, avec une pénalité de brièveté. Les scores obtenus sont modestes en raison du volume très limité du corpus d'entraînement (500 paires, 5 époques sur CPU) : un score BLEU-1 significatif nécessite au minimum 10 000 paires et 20 époques d'entraînement.

Les trois architectures ayant été étudiées indépendamment, le chapitre suivant propose une analyse transversale qui met en évidence leurs différences fondamentales et guide le choix de l'architecture en fonction de la nature des données.

5. Comparaison transversale des trois architectures

Ce chapitre synthetise les enseignements des trois parties du projet en proposant une comparaison structuree des architectures etudiees selon plusieurs dimensions : biais inductif, type de donnees privilegie, mecanisme de regularisation, et performance obtenue dans ce projet.

Tableau 6 : Comparaison multi-criteres : MLP, CNN et RNN/LSTM

Critere	MLP (Partie I)	CNN (Partie II)	RNN/LSTM (Partie III)
Type de donnees cible	Tabulaire / features	Images et signaux 2D	Sequences temporelles
Biais inductif	Aucun	Localite + inv. transl.	Ordre temporel
Partage de poids	Non	Oui (filtres conv.)	Oui (matrice Wh)
Memoire contextuelle	Aucune	Locale (champ recept.)	Globale (etat cache)
Regularisation cle	BN + Dropout + L2	BN + Dropout + Pooling	Dropout + Clip + TF
Entrainement	Backprop standard	Backprop standard	BPTT + grad clipping
Dataset utilise	Breast Cancer Wisc.	Fashion-MNIST	Shakespeare / Tatoeba
Performance principale	Acc. 98.84% (F1 99%)	Acc. 88-91%	PPL : LSTM < GRU < RNN

Le tableau precedent met en lumiere une progression logique : chaque architecture encode un biais inductif de plus en plus specialise. Le MLP n impose aucune contrainte sur la structure des donnees, ce qui le rend versatile mais moins efficace que des architectures adaptees lorsque la structure existe. Le CNN exploite la regularite spatiale des images via le partage de poids, reduisant le nombre de parametres d un facteur 3 tout en ameliorant les performances par rapport au MLP. Le LSTM ajoute une dimension temporelle explicite via son cell state, lui permettant de capturer des dependances longue distance impossibles a modeliser pour les deux architectures precedentes.

Guide de selection de l architecture

- MLP : donnees tabulaires structurees avec features numeriques ou categorielles, absence de structure spatiale ou temporelle, faible dimensionnalite. Exemple : prevision de prix, scoring de credit, diagnostic medical.
- CNN : donnees avec structure spatiale (images, spectrogrammes, signaux 1D courts), problemes ou l invariance a la translation est souhaitable. Exemple : classification d images, detection d objets, reconnaissance de parole.
- RNN/LSTM/GRU : sequences de longueur variable avec dependances temporelles ou contextuelles importantes. Exemple : NLP, traduction automatique, prediction de series temporelles, generation de texte.

Il est important de noter que ces trois architectures servent aujourd hui de briques fondamentales aux modeles Transformers (Vaswani et al., 2017), qui unifient et surpassent ces approches pour le traitement de sequences grace au mecanisme d attention multi-tete et a la parallelisation complete.

La conclusion du rapport revient sur les principaux enseignements du projet et propose des pistes d

amelioration pour chacune des trois parties.

6. Conclusion et perspectives

Ce projet a permis d'implémenter et d'évaluer de bout en bout trois familles fondamentales d'architectures de deep learning avec PyTorch, sur des jeux de données réels couvrant des domaines variés : le médical, la vision par ordinateur, et le traitement du langage naturel.

Les principaux enseignements peuvent être résumés comme suit. Le MLP s'avère une solution performante et robuste pour les données tabulaires bien structurées, atteignant une accuracy de 98.84% sur le dataset Breast Cancer Wisconsin grâce à une régularisation efficace (BatchNorm, Dropout, Early Stopping) et à l'initialisation Xavier. Le CNN exploite avec succès le biais inductif de localité et d'invariance spatiale propre aux images, surpassant le MLP avec trois fois moins de paramètres sur Fashion-MNIST. Enfin, les modèles récurrents illustrent comment encoder l'ordre temporel des séquences : le LSTM surpasse le RNN simple grâce à son cell state qui préserve le gradient sur de longues séquences, et le système Seq2Seq pose les bases de la traduction automatique neuronale.

Ces trois architectures s'inscrivent dans une progression historique et conceptuelle qui aboutit aux Transformers (Vaswani et al., 2017), architecture dominante en NLP et de plus en plus utilisée en vision (Vision Transformer, ViT). Comprendre les biais inductifs du MLP, du CNN et du LSTM est indispensable pour appréhender pourquoi l'attention multi-tête constitue un mécanisme plus général et plus puissant.

Perspectives d'amélioration

- Partie I : appliquer une analyse en composantes principales (ACP) pour réduire la multicollinéarité et simplifier le modèle sans perte de performance. Comparer avec un gradient boosting (XGBoost) pour quantifier l'apport du MLP.
- Partie II : explorer le transfer learning avec des modèles pré-entraînés (ResNet-18, EfficientNet) finetuned sur Fashion-MNIST pour dépasser les 93% d'accuracy. Implémenter la data augmentation (rotation, flip, crop) pour améliorer la robustesse.
- Partie III : ajouter un mécanisme d'attention de Bahdanau (2015) au Seq2Seq pour dépasser le goulot d'étranglement du vecteur contexte fixe. Entraîner sur un corpus plus grand (> 10 000 paires) pour obtenir un BLEU significatif.
- Global : expérimenter des optimiseurs récents (AdamW, Lion) et des schedulers cosinus avec warmup, qui se sont imposés comme standard dans les grandes architectures.

Repository GitHub : github.com/DEVBSOUL/projet-deep-learning

Technologies : Python 3.10+ | PyTorch | torchvision | scikit-learn | NumPy | Matplotlib | Seaborn